

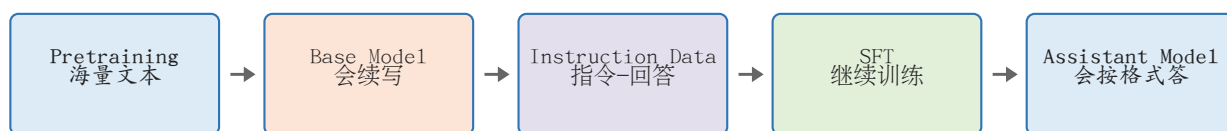
第16讲

Instruction Tuning：为什么预训练模型只是“会续写”， 微调以后才开始“会听指令”？

从 Base Model 到 Assistant Model: SFT、Chat Template、Loss Mask 与行为塑形

本讲总目标

真正理解：Instruction Tuning 并不是给模型“安装一个听话模块”，而是继续使用 next-token prediction，只是把训练数据改造成“指令/对话 → 理想回答”的形式，并让模型在这些示范上继续更新参数。



建议先记住一句话：

“预训练教模型语言世界的统计结构；Instruction Tuning 教模型在有人发出任务时，应该以什么行为方式作答。”

1. 先解决最核心的问题：Base Model

为什么不一定“听话”？

上一讲我们已经知道：预训练时，模型最基本的目标是根据已有 token 预测下一个 token。它看到的是互联网文章、书籍、代码、论坛讨论等连续文本。这个训练目标并没有天然规定：遇到一句“请解释牛顿第二定律”，模型就必须像助教一样先给定义、再举例、最后总结。

因此 Base Model 更像一个极强的“文本延续器”。它可能续写问题、模仿网页格式、继续生成另一个人的发言，也可能给出答案，但这种“回答用户”并不是预训练目标本身强制出来的唯一行为。

Base Model 的典型倾向	Assistant Model 的典型倾向
任务是继续文本 可能延续题目本身 格式未必稳定 不天然遵循 system/user/assistant 角色	把 user 视为任务发出者 产生面向用户的回答 更稳定遵循指令格式 学习拒答、风格和对话规范

通俗比喻	预训练像让一个人读完一座图书馆；Instruction Tuning 像再给他大量“老师提问—优秀学生回答”的示范，让他学会在被提问时应该怎样组织输出。
------	--

这并不意味着 Base Model “没有能力”。很多知识和语言能力在预训练阶段已经存在。Instruction Tuning 更多是在做行为接口的塑形（behavior shaping）：让已有能力更容易被自然语言指令调用。

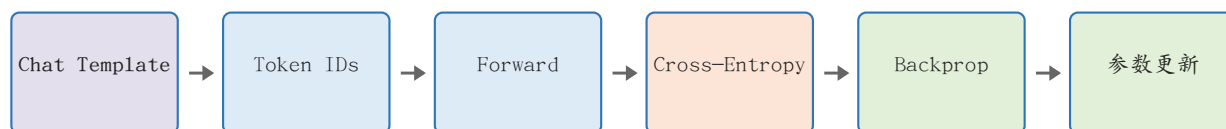
2. Instruction Tuning / SFT 到底是什么？

Instruction Tuning（指令微调）通常指：在一个已经完成预训练的语言模型上，再使用大量“指令 + 期望回答”样本进行监督训练。最常见的实现方式就是 SFT（Supervised Fine-Tuning，监督微调）。

注意：现实文献中“instruction tuning”和“SFT”有时会被较宽松地混用。严格说，SFT 是训练方式；Instruction Tuning 强调的是训练数据具有“任务指令/对话示范”的结构。

system	你是一名耐心、准确的助教。
user	请用零基础能听懂的话解释什么是过拟合。
assistant	过拟合可以理解为：模型把训练题“背得太熟”，以至于遇到新题时反而不会灵活处理……

模型会把上面整段对话编码成 token 序列。然后，它依然做我们已经熟悉的事情：



关键点	SFT 并没有把 GPT 的训练目标换成一个神秘的新算法。绝大多数情况下，它仍然是 next-token prediction + cross-entropy，只不过训练语料的分布和 loss 计算位置发生了变化。
-----	---

3. Chat Template：对话在模型眼里其实仍然只是 token

在用户界面里，我们看到的是 system、user、assistant 三种“角色”。但 Transformer 最终接收的仍然是一串 token。因此系统必须把结构化消息序列化成某种模板。不同模型的模板并不完全一样。

```
<bos>
<system> ■■■■■■ </system>
<user> ■■■■■■ </user>
<assistant> ■■■■■■..... </assistant>
<eos>
```

上面的尖括号只是教学示意，不代表任何具体商业模型的真实 token 格式。真实模型可能使用专门的 role tokens、分隔符、起止标记或其他协议。

System message: 给出高层行为规则、角色或约束。

User message: 当前任务、问题或输入。

Assistant message: 训练时的理想输出，也是模型需要学习生成的部分。

Special tokens: 用于标识角色边界、回合边界、结束位置等。

科研注意

比较不同模型时，不要忽略 chat template。即使底层模型相同，不同模板也可能改变行为。做 Agent 实验时，模板、system prompt、tool schema 都属于实验条件的一部分。

可以把 Chat Template 理解成 API 协议：它把“谁说了什么”编码成模型能处理的一维 token 序列。

4. SFT 中最容易忽略的关键：哪些 token 参与 Loss?

如果训练样本是“用户问题 + 助手答案”，一个常见做法是：用户部分只作为条件输入，不计算训练损失；助手答案部分才计算 loss。这通常通过 label masking 实现。

示意：

```
tokens : <user>  2 + 2 ?  <assistant>  4  <eos>
labels :   X    X X X X      X          4  <eos>
                X = ignore / masked
```

模型仍然会“看到”用户 prompt，因为这些 token 会进入前向计算并影响 hidden states；但训练时，我们通常不要求模型去学习复现 user 的问题本身，而重点要求它在 assistant 区域产生理想回答。

为什么这很重要？

如果把 user token 也当成主要训练目标，模型会花大量梯度去学习“复述用户问题/角色标记”，而不是把主要训练信号集中在如何回答上。具体实现仍会因训练框架和任务设计而不同。

从数学上看，SFT 可以写成只对回答 token 集合 A 求和：

$$L = - \sum_{t \in A} \log p(y_t \mid x, y_{<t})$$

这里 x 表示指令/上下文，y 表示助手回答。你现在应该能看出来：它依旧是 Cross-Entropy，只是求和位置受 mask 控制。

5. 多轮对话是怎么训练的？

多轮对话本质上只是更长的序列。比如：

system	你是一名数学助教。
user	什么是导数？
assistant	导数表示函数在某一点附近变化得有多快。

system	你是一名数学助教。
user	那梯度和导数是什么关系？
assistant	导数通常针对一元函数；梯度可以看成多元函数各个方向偏导数组成的向量。

实际训练时，历史回合可能全部被放进 context；loss 可以只计算最后一轮 assistant，也可以对多个 assistant 回合都计算，取决于具体 pipeline。

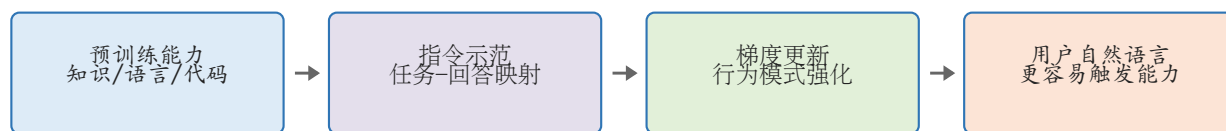
因此，多轮聊天能力并不是模型内部存在一个“聊天状态机”；很多时候，它首先来自：把历史对话放进上下文，再继续进行 next-token prediction。

和 Agent 的连接

Agent 框架里的“历史消息列表”最终也要被序列化进上下文。于是 Agent 记忆、工具结果、角色说明如果不断堆积，本质上都会占用 token，并改变下一步预测。

6. 为什么 Instruction Tuning 能让模型“突然变得会用”？

预训练阶段已经学到了大量潜在能力，但这些能力并不总是通过用户自然语言指令稳定地暴露出来。Instruction Tuning 的重要作用，是把“任务描述 → 期望行为”的映射变得更稳定。



可以用一个类比：一个人读过大量编程书并不等于他已经习惯“收到一句需求就给出可运行代码”。如果再让他看大量“需求 → 优秀实现”的示范，他就会学会这种交互协议。

它能强化回答格式、任务边界和角色行为。

它能让模型学会大量不同任务共享的“遵循自然语言指令”模式。

它可以让模型学会拒绝某些不允许的任务或遵循安全规范。

它通常不会凭空创造一个模型预训练阶段完全不具备的巨大知识世界。

不要误解

Instruction Tuning 不是“把知识库写进去”；很多知识主要来自预训练。SFT 更像是把知识和能力组织成更稳定、可调用的助手行为。

7. Instruction Dataset：什么样的示范数据才有价值？

SFT 的效果高度依赖示范数据。数据量重要，但数据质量、任务覆盖、答案风格和错误率同样重要。

维度	含义	可能问题
正确性	答案事实、推理和代码是否正确	错误答案会直接形成错误监督信号
多样性	覆盖问答、写作、代码、数学、工具等任务	任务过窄会导致行为偏置
难度	简单与复杂样本是否合理配比	全是简单题可能无法塑造复杂行为
风格	简洁/详细、格式、语气的一致性	模型会模仿示范中的表面风格
安全	对危险/不允许请求如何处理	边界示范不足会导致不稳定拒答

现代 instruction data 可以来自人工撰写、人工修订、已有任务数据、模型生成的 synthetic instructions、强模型蒸馏式示范等。不同来源的可靠性和许可条件不同。

科学视角	SFT 数据不是中性的“教材”。它定义了模型在什么情境下应该产生什么行为，因此数据配方本身就是模型行为设计的一部分。
------	--

8. Pretraining 和 SFT：到底哪些东西相同，哪些不同？

比较项	Pretraining	SFT / Instruction Tuning
起点	随机/初始参数或早期模型	已经完成预训练的 Base Model
数据	海量通用文本	指令-回答、对话示范
目标	学习语言与世界中的统计结构	学习按任务要求产生理想行为
核心 loss	通常 next-token cross-entropy	通常仍是 next-token cross-entropy
规模	通常极其庞大	通常远小于预训练数据
典型输出	Base Model	Instruction / Assistant Model

底层统一性

两阶段最令人容易忽略的地方是：底层 Transformer 仍然在做 token prediction。改变最大的不是“模型换了脑子”，而是它看到的数据分布、训练规模、loss mask 和行为目标发生了变化。

这也说明了为什么 Prompt Engineering 有效：模型已经在 SFT 中看过大量“自然语言任务描述 → 回答”的模式，因此新的 prompt 可以作为控制输入，激活某类行为。

9. SFT 能做什么？不能做什么？

SFT 擅长的	SFT 不能保证的
格式遵循 角色/语气 任务响应习惯 输出结构 工具调用格式示范 安全行为示范	事实永远正确 推理永远可靠 绝不幻觉 获得实时知识 自动拥有长期记忆 自动变成 Agent

例如，训练数据可以教模型在需要调用天气工具时输出某种 JSON 函数调用格式，但“真的去访问天气 API”仍然需要外部 runtime。

同理，SFT 可以强化“引用证据”的行为习惯，但如果上下文里没有可靠证据，模型仍可能生成看起来像引用的错误内容。

科研含义	看到 “instruction-tuned model 更好” 时，要问：是底层能力更强，还是任务接口更匹配？二者必须区分。对 Agent 研究尤其如此，因为角色 prompt、工具协议、chat template 都能显著改变行为。
------	---

10. SFT 也会失败：过拟合、能力偏移与灾难性遗忘

微调本质上仍然是在修改参数，因此如果数据太窄、学习率太大、训练太久，模型行为可能发生不希望的偏移。

Overfitting: 过度模仿训练样本措辞或模板，对新指令泛化变差。

Catastrophic Forgetting: 新训练把旧能力“冲淡”，某些通用能力下降。

Style Collapse: 回答格式变得过于统一，什么问题都套同一种模板。

Safety/Helpfulness Trade-off: 过度强化某种拒答或服从模式，可能伤害其他目标。

常见缓解思路包括：更好的数据混合、更小学习率、更少训练步、加入通用数据 replay、正则化、参数高效微调等。

重要直觉

微调不是“只新增知识、不动旧知识”。它修改的是同一套参数，因此不同能力之间可能发生干扰。

11. LoRA / QLoRA：为什么个人研究者也能做微调？

全参数微调（Full Fine-Tuning）会更新模型几乎所有权重，显存和计算开销很大。参数高效微调（PEFT）试图只训练很小一部分新增参数。

LoRA 的核心直觉可以写成：

$$W' = W + \Delta W, \text{ 且 } \Delta W \approx A \cdot B \text{ (低秩)}$$

原始大矩阵 W 冻结，只训练较小的 A 、 B 。这样可以显著减少可训练参数和优化器状态。

LoRA：冻结基础权重，学习低秩增量。

QLoRA：进一步把基础模型以低比特形式加载，同时训练 LoRA 适配器。

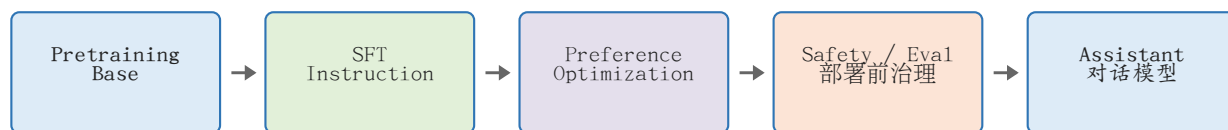
这使得“微调行为/领域格式”比从头训练大模型现实得多。

范围说明	本讲只建立概念。LoRA 的矩阵细节、rank、target modules、量化格式等适合在后续实验型课程中单独展开。
------	--

对于 Agent Systems 研究，这意味着你不必默认“研究 Agent 就必须训练基础模型”。大量实验可以直接用 API / 开源 instruct 模型；只有当问题本身涉及行为塑形、角色长期化或模型适配时，才需要考虑微调。

12. 从 Base Model 到 Chat Assistant: SFT 只是中间一站

现代助手模型通常不只经过一个阶段。一个高度简化的 pipeline 可以画成:



下一讲我们会继续讲 Preference Learning / RLHF / DPO。你可以先记住: SFT 提供“优秀示范”; 偏好优化进一步告诉模型“两个都像答案时, 人更喜欢哪一个”。

一句话区分

SFT: 模仿理想答案。 Preference Learning: 比较多个候选答案的好坏偏好。

因此“Chat 模型为什么行为与 Base 模型差很多”不能只归因于 SFT; 后续偏好优化、安全训练、系统提示和推理时策略也都会共同塑造最终行为。

13. Prompt、SFT、RAG：三种改变模型行为的方式不要混淆

方法	是否改参数	主要作用	典型场景
Prompt	否	利用当前上下文临时控制行为	指定格式、角色、任务
SFT / LoRA	是	把行为模式写入参数	稳定输出风格、领域任务适配
RAG	通常不改模型参数	运行时提供外部知识	私有文档、最新资料、证据检索

例如你希望模型掌握“学校今天刚更新的课程表”：最适合的是 RAG/工具读取，而不是重新 SFT；你希望模型长期稳定输出某种结构化诊断格式，则 SFT/LoRA 可能更合适；你只是临时让它“回答简洁一点”，一个 prompt 就够。

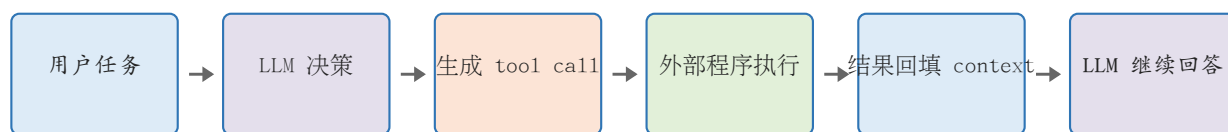
最小闭环能力	以后遇到一个“模型行为不符合需求”的问题，先判断：这是上下文控制问题（Prompt）、长期行为适配问题（SFT）还是外部知识问题（RAG）。不要一遇到问题就想到重新训练。
--------	---

14. Instruction Tuning 与 Tool Calling / Agent 有什么关系？

工具调用看起来像模型“会操作 API”，但从模型内部看，很多时候它仍然是在生成一种特定格式的 token。SFT 可以教它：当任务需要外部工具时，产生符合 schema 的函数调用。

```
user: ■■■■■■■■■■  
assistant(tool_call): {"name": "weather", "arguments": {"city": "■■■"}}  
tool: 18°C■■■■  
assistant: ■■■■■ 18°C■■■■
```

真正执行 `weather()` 的不是 Transformer 权重，而是 Agent runtime / tool executor。模型负责“决定要不要调用、生成参数、读取返回结果再继续生成”。



层级不要混淆

SFT 可以塑造“工具使用策略和调用格式”，但 Agent System 还包括运行时状态、工具权限、记忆、环境反馈、循环控制等系统组件。

15. 对 Agent Systems 科研来说，为什么这一讲很重要？

以后你比较不同 Agent 或不同多 Agent 协作机制时，模型的“instruction-tuned 状态”本身就是一个重要实验变量。

同一个 Base Model 和它的 Instruct 版本，行为可能明显不同。

不同 chat template 会改变系统提示和角色信息如何进入模型。

Agent role prompt 可能与模型 SFT 中学到的对话模式产生强交互。

工具调用能力可能来自专门 SFT，而不是“基础推理能力”本身。

多 Agent 实验中的“人格/角色差异”有时只是 prompt-induced heterogeneity，不等于模型内部形成了长期功能异质性。

因此，如果你以后研究“长期经历导致 Agent 异质性”，要区分至少三类来源：

异质性来源	是否改参数	例子
Prompt / Persona	否	你是审稿人 / 你是工程师
Memory / Experience	通常否（外部状态）	不同 Agent 保存不同经历与反馈
Fine-tuning	是	不同 Agent 经过不同 LoRA/SFT 数据训练

16. 初学者最常见的 10 个误区

1. “SFT 会把模型变成一个全新模型。”

通常架构不变，主要是参数继续被更新，行为分布发生变化。

2. “指令微调以后模型就不会幻觉。”

SFT 可以塑造回答风格，但不能保证事实真实性。

3. “Base Model 不懂问题，Instruct Model 才懂。”

Base 模型可能已有大量能力；SFT 主要改善自然语言任务接口和行为稳定性。

4. “ChatGPT 类助手就是一个纯 SFT 模型。”

最终行为还可能受偏好优化、安全训练、系统提示、工具/runtime 等影响。

5. “System Prompt 是写进参数里的。”

系统提示通常是推理时上下文，不等于参数微调。

6. “RAG 就是微调。”

RAG 通常在运行时检索并注入上下文，不必修改参数。

7. “LoRA 不会改变模型行为。”

LoRA 虽只训练小量参数，但这些增量足以显著改变输出分布。

8. “微调数据越多越好。”

质量、覆盖、重复度和训练步数都重要，过多低质数据可能伤害模型。

9. “工具调用意味着模型真的执行了工具。”

模型通常只生成调用请求，外部程序负责真正执行。

10. “两个模型回答风格不同就说明能力不同。”

可能只是 SFT、模板或系统提示不同，需控制变量。

17. 小实验：亲手看懂 Chat Template 与 SFT Loss Mask

材料包中的 04_sft_loss_mask_demo.py 不训练真正的大模型，而是完成三个最关键的教学动作：

- 1. 把 system / user / assistant 消息序列化成一个 toy chat template。
- 2. 把序列切成 token，并构造 labels。
- 3. 把 system + user 部分设为 ignore，只让 assistant 回答 token 贡献 cross-entropy loss。

程序还会给出一组 toy probability，比较“回答 token 概率提高”前后的平均 loss，让你看到 SFT 的优化方向。

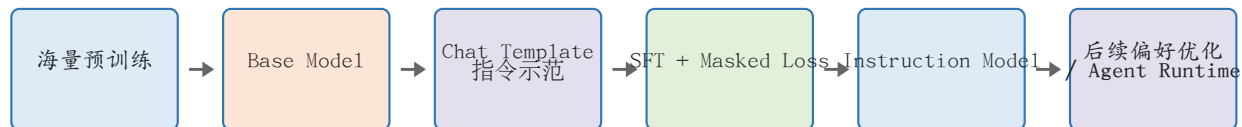
Token	Role	Label

<system>	system	IGNORE
...	...	IGNORE
<assistant>	control	IGNORE
■■	assistant	■■
■	assistant	■
...	assistant	...

实验目的

不是模拟真实 LLM 的全部训练，而是把“输入仍然整段可见，loss 只在回答区域计算”这个最重要的机制变成可观察的数字。

18. 本讲总结：把整条链压缩成一张图



请至少掌握下面 8 句话：

1. Base Model 的核心训练目标是文本续写，不天然等价于“助手行为”。
2. Instruction Tuning 通常是在预训练模型上继续做监督微调。
3. 底层常常仍是 next-token prediction + cross-entropy。
4. Chat Template 把 role/message 序列化 token。
5. SFT 常用 label mask，让 assistant 回答区域承担主要 loss。
6. SFT 更像行为接口塑形，不保证事实正确，也不等于外部知识更新。
7. Prompt、SFT、RAG 分别对应临时上下文控制、参数级行为适配、运行时外部知识。
8. Agent 的工具调用可以通过 SFT 学格式，但真正执行工具的是系统 runtime。

进入下一讲

下一讲讲 Preference Learning / RLHF / DPO：当“有一个标准答案可模仿”不够时，如何让模型学会“两个答案哪个更符合人的偏好”。

19. 课后自检（答案在独立练习册）

为什么 Base Model 会“续写问题”而不一定直接回答？

Instruction Tuning 和 Pretraining 最关键的相同点是什么？不同点是什么？

为什么 system/user token 可能不参与 loss，却仍然会影响回答？

Chat Template 为什么可能成为科研实验中的混杂变量？

SFT 能否让模型自动获得实时新闻？为什么？

Prompt、SFT、RAG 应该如何选择？

Tool Calling 中模型和 runtime 分别负责什么？

为什么“角色 prompt 导致的异质性”不能直接等同于“长期经历形成的功能性异质性”？

建议学习顺序：先通读本讲 → 运行代码 → 做练习 → 再回看第13讲

Pretraining。你会发现 Pretraining 和 SFT 的底层训练机制其实高度统一。